

수업 04 · 전체 과정 8/20일차

Git/GitHub

2026-10-16 (금) · 10:00~17:00

50분 수업 · 10분 휴식 · 중식 11:50~13:00

파일의 변경을 비교하고 Git으로 기록합니다. 로컬 저장소 실습과 제공된 GitHub 리뷰 사례로 협업 흐름을 익힙니다.

교안 읽기	직접 해보기	결과 확인하기
개념과 완성 예제를 먼저 봅니다.	필수 활동을 열어 작성·수정합니다.	예상과 실제를 비교하고 설명합니다.

처음 배우는 사람을 위한 수업

파일과 폴더, 저장과 실행부터 설명합니다. 설치·계정 문제가 있으면 제공 예제의 수동 추적이나 강사 시연으로 같은 개념을 확인하고, 실제 실행 여부를 구분해 기록합니다.

웹 강의실 열기 · 화면의 링크와 내려받기 자료를 함께 사용하세요.

PDF 제작일 2026-10-05 · 원본 웹 내용 + 50분 교시 운영·설명 보완

이 교안의 순서

오늘 준비할 것	3
오늘의 6교시	5
1교시 · 작업 폴더와 저장소	6
2교시 · 선택한 변경을 기록하기	12
3교시 · 기록의 의미와 관리 대상	17
4교시 · 비교와 범위 있는 복구	23
5교시 · 공유 저장소와 리뷰	28
6교시 · 협업 사례와 인계	32
추가 활동 · 필수 경로를 마친 뒤	38
용어와 코드 기호 찾아보기	42
오늘 배운 내용 확인하기	43
파일과 다음 수업	45

오늘 준비할 것

Git/GitHub · 변경을 비교하고 기록하기

2026-10-16 · Git/GitHub

파일의 변경을 비교하고 Git으로 기록합니다. 로컬 저장소 실습과 제공된 GitHub 리뷰 사례로 협업 흐름을 익힙니다.
원본은 그대로 두고 work 폴더의 지정한 파일을 수정하세요. 계정 없이 사용할 수 있는 연습용 저장소로 진행합니다.

오늘의 연결

지금까지 만든 파일의 변경을 비교하고 이전 상태를 기록하는 방법을 배웁니다.

먼저 구별할 화면

- 파일 탐색기는 저장한 파일의 위치를 찾습니다.
- 텍스트 편집기는 코드와 데이터를 읽고 고칩니다.
- 터미널은 명령을 입력하고 실제 실행 결과를 확인합니다.
- 웹 교안은 설명·원문·출력 예를 제공합니다. Git 명령을 실행하는 터미널은 아닙니다.

참여 순서

1. 압축을 일반 폴더에 풀고 data-code-work의 위치를 유지합니다.
2. lesson의 개념을 읽고 activity-guide의 해당 활동 입력을 엽니다.
3. work의 답칸에 예상과 이유를 먼저 쓰고 지정한 값을 수정합니다.
4. 저장 후 정확한 work 파일을 실행하거나 강사 출력·제공 예와 수동 비교합니다.
5. 확인 방식을 기록합니다. 개인 실행을 하지 않았으면 실행 성공으로 적지 않습니다.

도구를 준비하지 못했을 때

설치 문제로 수업을 멈추지 말고 제공한 전체 코드·출력과 동일 입력을 사용해 줄별 값을 직접 추적합니다. 종이에도 같은 답칸을 작성합니다. 개인 실행은 강사가 환경을 준비한 뒤 확인합니다.

[교안](#) · [실습](#) · [용어](#)

수업 전, 이 세 가지부터 확인하세요

앞선 수업을 기억하지 못해도 아래 질문과 풀이부터 다시 읽으면 됩니다. 먼저 자기 말로 답한 뒤 풀이와 비교하세요.

저장과 기록

질문: 파일을 저장하면 Git 이력도 자동으로 생기나요?
풀이: 저장은 작업 파일을 바꿉니다. 변경 확인, add, commit을 거쳐 기록을 남기는 절차는 별개입니다.

내 컴퓨터와 공유 위치

질문: 로컬 commit 후 다른 사람도 바로 그 내용을 받나요?
풀이: 원격 저장소에 보내는 과정은 별도입니다. 로컬 기록과 원격 반영을 구별해 확인합니다.

충돌

질문: 두 사람이 같은 줄을 바꾸면 최신 파일로 덮으면 될까요?
풀이: 두 변경의 의도를 비교해 결과를 결정하고 검증해야 합니다. 단순 덮어쓰기는 필요한 변경을 잃을 수 있습니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02, A03	새 입력으로 한 번 더 확인
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11	A12
5	A13, A15	A14
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 쉬운 사례와 단계별 풀이를 먼저 읽고 필수 결과를 직접 만들어 봅니다. 새 독립 문제는 결과를 확인한 뒤 복습하거나 도전합니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

오늘의 6교시

교시·시간	학습 초점	필수 활동
1교시 10:00~10:50	저장과 변경 기록	A01, A02, A03
2교시 11:00~11:50	비교·선택·기록	A04, A05, A06
3교시 13:00~13:50	기록의 의미와 관리 대상	A07, A08, A09
4교시 14:00~14:50	선택 취소와 내용 복구	A10, A11
5교시 15:00~15:50	공유 저장소와 검토 요청	A13, A15
6교시 16:00~16:50	충돌 의미 결정과 인계	A16, A17, A18

설명·시연·함께 수행·개별 수행·풀이를 합쳐 교시당 50분입니다. 추가 활동까지 모두 같은 시간에 끝내라는 뜻은 아닙니다. 마지막 10분은 휴식·종료 준비로 배치합니다. 실제 학습자 소요시간은 수업에서 측정해 조정합니다.

2026-10-16 · Git/GitHub

파일의 변경을 비교하고 Git으로 기록합니다. 로컬 저장소 실습과 제공된 GitHub 리뷰 사례로 협업 흐름을 익힙니다.

최신 운영은 50분 × 6교시, 순수 수업 300분입니다. 설명 보충은 필수 개념이고 독립 과제는 추가 복습입니다. 보충 전체를 50분에 완수했다는 뜻은 아니며 실제 학습자 리허설 전입니다. 원본을 보존하고 work의 지정한 부분을 수정합니다. 계정이나 실제 장비 없이 합성 자료로 진행합니다.

1교시 · 작업 폴더와 저장소

10:00~10:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 저장과 변경 기록

먼저 할 활동: A01, A02, A03

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A01 · 파일 저장과 변경 기록

사례로 이해하기 · 파일 저장에서 변경 이력까지

실무 상황

안내 담당자가 센서 설명서의 실행 위치를 수정했습니다. 다음 날 동료가 “어제 안내와 무엇이 달라졌나요?”라고 묻습니다. 현재 파일만 있으면 기억에 의존하지만 변경 기록이 있으면 두 문장을 나란히 볼 수 있습니다. 파일 이름에 최종을 여러 번 붙이는 대신 같은 파일의 상태를 차례로 기록합니다.

쉬운 풀이

작업 영역은 지금 편집하는 종이이고, 스테이징은 이번 회의에 제출할 종이를 골라 둔 봉투이며, 커밋은 제출한 봉투의 내용을 번호와 설명으로 보관한 기록입니다. 봉투를 고른 뒤 종이를 다시 고치면 봉투의 사본은 저절로 바뀌지 않습니다. 저장과 선택과 기록이 세 행동인 이유입니다. Git은 이 과정을 관리하는 도구이며 GitHub는 기록을 공유할 수 있는 서비스입니다.

완성 예와 풀이

README.md를 만든 뒤 저장하고 `git status --short`를 읽으면 처음에는 `?? README.md`입니다. `git add README.md` 뒤에는 `A README.md`가 됩니다. `git commit -m "Add run instructions"`를 실행하면 첫 기록이 생깁니다. 상태 확인은 조회, add는 선택, commit은 기록입니다. 오늘 실습 저장소의 위치를 먼저 확인하고 이 순서를 A02~A06에서 실행합니다. 이 예에는 인터넷이나 GitHub 로그인이 필요하지 않습니다.

독립 추가 복습 과제

안내문과 개인 메모라는 두 파일이 있습니다. 안내문만 동료에게 인계할 때 각 파일을 어느 상태까지 진행할지 그림으로 작성하세요. 다음으로 저장만 한 안내문을 내일 다시 찾을 수는 있지만 어제 내용과 비교하기 어려운 이유를 설명하세요.

답과 대표 오답

안내문은 저장·선택·커밋으로 기록하고 개인 메모는 선택하지 않습니다. “파일이 저장됐으니 GitHub에도 있다”는 답은 공유 단계가 빠졌습니다. “status를 실행했으니 이력이 생겼다”도 조회와 변경을 혼동합니다.

확인 질문과 해설

① 인터넷이 없을 때 무엇이 가능한가요? 로컬 저장·선택·커밋이 가능합니다. ② add 뒤 새 문장을 저장하면 무엇을 다시 확인하나요? 작업 영역과 스테이징의 차이를 확인합니다. ③ 기록의 번호가 교안과 달라지면 실패인가요? 환경과 기록 내용에 따라 번호가 달라지므로 메시지와 파일 내용을 확인합니다.

개념. 파일 저장은 현재 내용을 디스크에 쓰는 행동입니다. Git 커밋은 선택한 파일 상태에 설명을 붙여 변경 이력으로 남기는 행동입니다. 최신 파일만 보관하는 것과 과거 상태를 비교할 수 있는 기록은 다릅니다.

왜 필요한가. 최종·진짜최종 같은 파일명을 계속 만들면 어떤 변경이 어느 목적이었는지 찾기 어렵습니다. 같은 파일의 변경을 기록하면 이전 내용과 현재 내용을 비교할 수 있습니다.

함께 보는 예. README의 안내를 바꿔 저장한 직후에는 편집한 파일은 달라졌지만 새 커밋이 생기지는 않습니다. 나중에 선택·기록 단계를 거쳐야 그 변경이 이력에 남습니다.

헷갈리는 경우. Git을 설치했다고 모든 폴더가 자동으로 추적되지는 않습니다. 저장소를 만들고 관리할 파일을 선택해야 합니다. GitHub 로그인도 로컬 변경 기록의 필수 조건은 아닙니다.

제공 입력

카드: 파일편집 / 저장 / git add / git commit / GitHub에보내기.

답칸: 어떤 상태가 바뀌나 / 로컬에서 가능한가 / 새 커밋이 생기나.

A01 · 따라 하기와 작성칸

A01 · 파일 저장과 변경 기록

목적: 최종·진짜최종 같은 파일명을 계속 만들면 어떤 변경이 어느 목적이었는지 찾기 어렵습니다. 같은 파일의 변경을 기록하면 이전 내용과 현재 내용을 비교할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A01_input.md](#)

- [A01_notes.md](#)

단계

1. 카드를 순서대로 놓고 각 행동의 결과를 씁니다.
2. 저장만 한 경우와 커밋한 경우를 구별합니다.
3. 오늘 수정할 README의 목적을 읽습니다.

4. 현재 컴퓨터와 공유 저장소의 역할을 다른 칸에 적습니다.

예상 결과: 저장과 커밋, 로컬 기록과 공유를 구별합니다.

확인 방법: 인터넷이 끊겨도 로컬 커밋을 만들 수 있나요?

오류 해결: Git과 GitHub가 섞이면 내 컴퓨터의 기록 도구와 웹 서비스라는 역할부터 나눕니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A02 · 연습 저장소와 로컬 작성자 설정

개념. git init은 지정한 폴더를 Git 저장소로 시작합니다. .git에는 변경 기록과 설정이 들어 있습니다. 작성자 이름과 이메일은 커밋에 남는 정보이며 로그인 암호가 아닙니다.

왜 필요한가. 실제 프로젝트와 분리한 연습 폴더를 사용해야 명령의 범위를 분명히 볼 수 있습니다. 설정은 --local로 이번 저장소에만 적용하며 컴퓨터 전체 설정을 바꾸지 않습니다.

함께 보는 예. course의 work 아래 git-lab을 만들고 그 안에서 시작합니다. main은 첫 브랜치 이름입니다. user.email은 실습용 .invalid 주소를 쓰며 실제 GitHub 계정 인증을 수행하지 않습니다.

헛갈리는 경우. 상위 KDigital_PhysicalAI 프로젝트에서 init-restore를 실행하지 않습니다. 매번 현재 위치와 git rev-parse 결과로 연습 저장소 경계를 확인합니다.

전체 명령

```
New-Item -ItemType Directory -Path work/git-lab

Set-Location work/git-lab

git init -b main

git config --local user.name "실습자"

git config --local user.email "learner@example.invalid"

git rev-parse --show-toplevel
```

각 줄의 뜻

줄	코드	뜻
1	<code>New-Item -ItemType Directory -Path work/git-lab</code>	연습 전용 폴더를 만듭니다.
2	<code>Set-Location work/git-lab</code>	그 폴더로 현재 위치를 옮깁니다.
3	<code>git init -b main</code>	main으로 시작하는 로컬 저장소를 만듭니다.
4	<code>git config --local user.name "실습자"</code>	이번 저장소의 작성자 이름을 정합니다.
5	<code>git config --local user.email "learner@example.invalid"</code>	실습용 작성자 주소를 정합니다.
6	<code>git rev-parse --show-toplevel</code>	저장소 최상위 주소가 git-lab인지 확인합니다.

실행 위치와 확인

처음 위치는 압축을 푼 course 폴더입니다. Windows PowerShell 예시이며 다른 OS의 폴더 생성·이동은 강사 안내를 따릅니다. git-lab이 이미 있으면 새 복사본의 빈 폴더를 사용합니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
Initialized empty Git repository in .../work/git-lab/.git/
.../work/git-lab
```

A02 · 따라 하기와 작성칸

A02 · 연습 저장소와 로컬 작성자 설정

목적: 실제 프로젝트와 분리한 연습 폴더를 사용해야 명령의 범위를 분명히 볼 수 있습니다. 설정은 --local로 이번 저장소에만 적용하며 컴퓨터 전체 설정을 바꾸지 않습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A02_input.md](#)

- [A02_setup.txt](#)

- [A02_notes.md](#)

- [A02_setup.txt](#)

단계

1. 일반 폴더로 압축을 풉니다.

- PowerShell의 현재 위치가 course인지 확인합니다.
- 명령을 한 줄씩 실행하며 작성자 설정의 범위를 설명합니다.
- 최상위 주소 끝이 work/git-lab인지 기록합니다.

예상 결과: 연습 폴더 안의 .git 생성과 로컬 설정. 아직 커밋은 없습니다.

확인 방법: 이메일 설정은 GitHub 로그인인가요?

오류 해결: git 명령이 없으면 강사가 설치 상태를 확인합니다. 제공한 명령·상태 카드로 수동 추적하고 개인 실행은 미완료로 표시합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A03 · status로 현재 상태 읽기

개념. git status는 현재 작업 파일과 스테이징 상태를 보여 줍니다. 처음 만든 파일의 ??는 아직 Git이 추적하지 않는 파일이라는 짧은 표시입니다.

왜 필요한가. 파일이 폴더에 존재한다는 사실과 이력에 들어갔다는 사실을 구별해야 합니다. 명령을 실행한 뒤 상태를 읽으면 어느 단계까지 했는지 확인할 수 있습니다.

함께 보는 예. data/initial_README.md의 내용을 탐색기로 git-lab/README.md에 복사하면 아직 추적하지 않은 파일로 나타납니다. status만 보는 것은 기록을 만들거나 파일을 바꾸지 않습니다.

헷갈리는 경우. working tree clean이라는 문구는 현재 비교할 변경이 없다는 뜻입니다. 코드가 기능적으로 올바르거나 GitHub에 올라갔다는 뜻은 아닙니다.

전체 명령

```
git status
git status --short
```

각 줄의 뜻

줄	코드	뜻
1	<code>git status</code>	자세한 현재 상태를 읽습니다.
2	<code>git status --short</code>	같은 상태를 짧은 기호로 확인합니다.

실행 위치와 확인

현재 위치: work/git-lab. 먼저 원자료 initial_README.md를 README.md로 복사하여 저장합니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
?? README.md
```

A03 · 따라하기와 작성칸

A03 · status로 현재 상태 읽기

추가 원문·작업 파일:

- [A03_input.md](#)
- [A03_status.txt](#)
- [initial_README.md](#)
- [A03_notes.md](#)
- [A03_status.txt](#)

목적: 파일이 폴더에 존재한다는 사실과 이력에 들어갔다는 사실을 구별해야 합니다. 명령을 실행한 뒤 상태를 읽으면 어느 단계까지 했는지 확인할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A03_input.md](#)
- [A03_status.txt](#)
- [A03_notes.md](#)
- [A03_status.txt](#)

단계

1. initial_README.md를 새 README.md로 복사합니다.
2. 상태 명령 두 가지를 비교합니다.
3. ??의 의미와 아직 하지 않은 행동을 씁니다.
4. 파일 내용과 변경 상태가 별개임을 설명합니다.

예상 결과: README가 untracked 또는??로 표시됩니다.

확인 방법: status를 열 번 실행하면 커밋이 생기나요?

오류 해결: README가 안 보이면 저장한 경로와 파일 확장자를 확인합니다. 다른 폴더의 파일을 보고 있을 수 있습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

2교시 · 선택한 변경을 기록하기

11:00~11:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 비교·선택·기록

먼저 할 활동: A04, A05, A06

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A04 · 작업 변경과 diff의 기준

사례로 이해하기 · 세 비교 기준으로 커밋할 내용을 판단하기

실무 상황

README 안내를 “course에서 실행”으로 바꾸어 선택한 뒤, 담당자가 다시 “압축을 푼 course에서 실행”으로 수정했습니다. 지금 커밋하면 가장 최근 화면 문장이 들어갈까요? 작업 파일을 보고 답하지 말고 다음 기록에 선택된 사본을 확인해야 합니다.

쉬운 풀이

비교는 항상 두 대상을 필요로 합니다. 기본 `git diff`는 작업 영역과 스테이징을 비교하고 `git diff --staged`는 스테이징과 마지막 커밋을 비교합니다. `git diff HEAD`는 현재 작업 영역과 마지막 커밋을 비교합니다. HEAD는 지금 브랜치에서 가리키는 마지막 기록입니다. 초심자는 명령을 외우기 전에 “왼쪽 대상과 오른쪽 대상”을 말로 적습니다.

완성 예와 단계 풀이

마지막 기록이 `Run from here.`라고 가정합니다. 작업본을 `Run from course.`로 저장하고 `git add README.md`를 합니다. 다시 `Run from extracted course.`로 저장합니다. `git status --short`의 `MM README.md`는 선택된 변경과 아직 선택되지 않은 변경이 둘 다 있다는 뜻입니다. 기본 diff에는 `course`와 `extracted course`의 차이가, `staged diff`에는 `here`와 `course`의 차이가 나옵니다. 지금 커밋하면 `course` 문장이 기록됩니다. 최신 문장을 넣으려면 내용을 읽고 다시 `add`하여 `staged diff`를 확인합니다.

독립 추가 복습 과제

세 칸에 각각 “커밋: 온도 28”, “선택: 온도 30”, “작업: 온도 29”를 적습니다. 각 비교 명령에 나타날 이전 값과 새 값을 직접 작성하고 지금 커밋할 값도 결정하세요. 요구가 29라면 무엇을 추가로 해야 할지 설명하세요.

답과 대표 오답

기본 diff는 30→29, `staged diff`는 28→30, HEAD 비교는 28→29입니다. 현재 커밋할 값은 30이며 요구가 29면 다시 선택합니다. “MM은 파일이 두 개”는 오답입니다. 한 파일의 서로 다른 두 비교 관계를 표시합니다.

확인 질문과 해설

① diff가 비면 변경이 없나요? 비교 기준 밖 변경이나 추적되지 않은 파일이 있을 수 있습니다. ② 추가 기호를 문장에 넣나요? 비교 표시이므로 최종 파일에는 넣지 않습니다. ③ 커밋 직전 가장 직접적인 확인은 무엇인가요? staged diff로 기록할 내용을 읽는 것입니다.

개념. diff는 두 상태에서 추가·삭제된 줄을 보여 줍니다. 기본 git diff는 작업 영역과 스테이징을 비교합니다. +는 추가된 줄, -는 빠진 줄을 뜻하며 최종 파일에 이 기호를 직접 넣는 것은 아닙니다.

왜 필요한가. 기록하기 전에 의도한 변경만 있는지 검토해야 합니다. 상태는 파일 수준을, diff는 줄 수준의 차이를 보여 줍니다.

함께 보는 예. 아직 추적하지 않은 파일은 기본 diff에 나타나지 않을 수 있습니다. 첫 파일은 add 뒤 --staged로 비교합니다. 이후 추적된 README를 수정하면 기본 diff에서 바뀐 줄을 읽을 수 있습니다.

헛갈리는 경우. diff가 비었다고 파일이 없거나 내용이 틀리지 않았다는 결론은 아닙니다. 비교 기준과 추적 상태를 함께 확인합니다.

제공 입력

비교 카드:

- 1 새 README는??이고 git diff가 비어 있음.
 - 2 추적 README의 'Run checker.'를'Run checker from course.'로 수정.
- 삭제줄 -Run checker.
- 추가줄 +Run checker from course.

A04 · 따라 하기와 작성칸

A04 · 작업 변경과 diff의 기준

목적: 기록하기 전에 의도한 변경만 있는지 검토해야 합니다. 상태는 파일 수준을, diff는 줄 수준의 차이를 보여 줍니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A04_input.md](#)

- [A04_notes.md](#)

단계

1. 각 카드의 비교 대상 두 상태를 적습니다.
2. 추적되지 않은 파일과 추적 파일을 구별합니다.
3. 삭제·추가 기호를 최종 문장으로 재구성합니다.

4. 의도한 한 문장 변경만 있는지 검토합니다.

예상 결과: 상태와 diff를 함께 읽고 비교 기준을 설명합니다.

확인 방법: 기본 diff는 항상 마지막 커밋과 비교하나요?

오류 해결: 변경이 안 보이면 status와 --staged 여부를 확인합니다. 같은 파일명의 다른 복사본을 고쳤는지도 봅니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A05 · add는 현재 변경을 선택한다

개념. git add는 지금의 파일 내용을 다음 커밋에 넣을 상태로 선택합니다. add 뒤 파일을 다시 고쳐도 스테이징이 자동으로 최신 내용으로 바뀌지는 않습니다.

왜 필요한가. 기록할 묶음을 고르고 검토해야 의도한 변경만 커밋할 수 있습니다. 'add 했으니 이후 모든 저장도 기록된다'는 오해를 막습니다.

함께 보는 예. 처음 README를 add하면 짧은 상태의 왼쪽 A가 추가 준비를 나타냅니다. --staged는 선택한 상태와 마지막 커밋을 비교합니다. 첫 커밋 전에는 선택한 전체 내용을 볼 수 있습니다.

헷갈리는 경우. git add .는 여러 파일을 한꺼번에 고를 수 있습니다. 초반에는 README.md처럼 이름을 명시해 무엇을 선택했는지 이해합니다.

전체 명령

```
git add README.md
git status --short
git diff --staged
```

각 줄의 뜻

줄	코드	뜻
1	<code>git add README.md</code>	README의 현재 상태를 선택합니다.
2	<code>git status --short</code>	선택 상태를 짧게 읽습니다.
3	<code>git diff --staged</code>	이번 기록에 들어갈 내용을 비교합니다.

실행 위치와 확인

현재 위치: work/git-lab. README가 저장되어 있고 첫 커밋 전 상태입니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

A README.md

선택한 README 전체 내용이 추가 줄로 보입니다.

A05 · 따라 하기와 작성칸

A05 · add는 현재 변경을 선택한다

목적: 기록할 묶음을 고르고 검토해야 의도한 변경만 커밋할 수 있습니다. 'add 했으니 이후 모든 저장도 기록된다'는 오해를 막습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A05_input.md](#)

- [A05_stage.txt](#)

- [A05_notes.md](#)

- [A05_stage.txt](#)

단계

1. README 한 파일만 add합니다.
2. 상태 왼쪽 표시와 staged 차이를 읽습니다.
3. 지금 커밋하면 들어갈 문장을 적습니다.
4. add 뒤 편집한 가상 카드에서 스테이징과 작업 영역을 나눕니다.

예상 결과: 선택한 파일은A로 표시되고 staged 비교에 내용이 나타납니다.

확인 방법: add 뒤 고친 내용까지 넣으려면 무엇을 확인해야 하나요?

오류 해결: staged에 예상 밖 내용이 있으면 커밋 전에 파일 경로와 비교를 확인합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A06 · commit과 변경 이력

개념. 커밋은 스테이징한 상태를 메시지와 함께 이력에 남깁니다. 커밋에는 식별 해시와 작성자·메시지 등이 있으며 해시 값은 각 실습 환경에서 달라질 수 있습니다.

왜 필요한가. 무엇을 왜 바꿨는지 나중에 확인할 수 있도록 기록합니다. 파일 저장→선택→기록의 세 단계를 직접 수행합니다.

함께 보는 예. 첫 README를 'Add checker guide'라는 메시지로 기록합니다. 로그에서 짧은 해시와 메시지를 찾고 status에서 남은 변경을 확인합니다.

햇갈리는 경우. commit은 GitHub로 보내는 명령이 아닙니다. 로컬 기록이 생겨도 공유 저장소에 전송하려면 별도 작업이 필요합니다.

전체 명령

```
git commit -m "Add checker guide"
git log --oneline
git status --short
```

각 줄의 뜻

줄	코드	뜻
1	<code>git commit -m "Add checker guide"</code>	선택한 상태를 메시지와 함께 기록합니다.
2	<code>git log --oneline</code>	짧은 커밋 이력을 봅니다.
3	<code>git status --short</code>	기록 뒤 남은 변경이 있는지 확인합니다.

실행 위치와 확인

현재 위치: work/git-lab. A05에서 README를 스테이징했습니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
<짧은해시> Add checker guide
status --short는 남은 변경이 없으면 빈 출력입니다.
```

A06 · 따라 하기와 작성칸

A06 · commit과 변경 이력

목적: 무엇을 왜 바꿨는지 나중에 확인할 수 있도록 기록합니다. 파일 저장→선택→기록의 세 단계를 직접 수행합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A06_input.md](#)

- [A06_commit.txt](#)

- [A06_notes.md](#)

- [A06_commit.txt](#)

단계

1. 선택한 내용을 마지막으로 확인합니다.
2. 첫 커밋을 만듭니다.
3. 자신의 실제 해시와 메시지를 기록합니다.
4. 아직 수행하지 않은 공유 단계를 표시합니다.

예상 결과: 첫 커밋 한 개. 남은 변경이 없으면 상태가 깨끗합니다.

확인 방법: 커밋 해시가 교안과 달라지면 실패인가요?

오류 해결: 작성자 오류는 로컬 user.name과user.email 설정을 확인합니다. 글로벌 설정을 임의로 바꾸지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

3교시 · 기록의 의미와 관리 대상

13:00~13:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 기록의 의미와 관리 대상

먼저 할 활동: A07, A08, A09

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A07 · 한 목적의 변경과 메시지

사례로 이해하기 · 한 목적의 기록과 비밀 파일의 경계

실무 상황

실행 안내 수정, 제목 오타 수정, 개인 접속 정보 추가가 같은 폴더에서 발생했습니다. 모두 한 번에 기록하면 동료는 어떤 변경을 검토해야 하는지 찾기 어렵습니다. 나중에 안내만 되돌리고 싶어도 다른 변경과 얽힙니다. 업무 목적이 같은 변경끼리 묶고 공유하면 안 되는 자료를 제외합니다.

쉬운 풀이

커밋 메시지는 기록의 표제입니다. “수정”보다 “README에 실행 폴더 안내 추가”처럼 대상과 이유를 적으면 검색할 수 있습니다. .gitignore는 새 파일 가운데 보통 추적하지 않을 이름을 정하는 목록입니다. 과거 기록을 지우는 지우개가 아니며 이미 추적하는 파일에는 기대한 방식으로 적용되지 않습니다. 비밀을 이름만 숨겼다고 안전한 이력이 되는 것은 아닙니다.

완성 예와 풀이

README.md의 실행 안내를 수정하고 `git add README.md`, `git diff --staged`, `git commit -m "Explain course run folder"` 순서로 확인합니다. 연습의 빈 `.gitignore`에 `local-note.txt`를 적은 뒤 가짜 개인 메모를 같은 이름으로 만듭니다. `git check-ignore local-note.txt`는 적용되는 이름을 보여 줍니다. `git ls-files`는 이미 추적한 목록을 보여 줍니다. 두 결과의 질문이 다르므로 하나만 보고 비밀 파일의 공유 여부를 단정하지 않습니다. 실제 비밀번호는 실습에 쓰지 않습니다.

독립 추가 복습 과제

README 실행 위치와 센서 기준값을 동시에 바꾼 상황에서 커밋 메시지 두 개를 작성하고 변경을 나눌 이유를 설명하세요. 이미 커밋한 가짜 `secret.txt`를 나중에 `ignore` 목록에 넣으면 과거 이력이 어떻게 되는지도 예상하세요.

답과 대표 오답

실행 안내와 센서 기준은 목적과 검증이 다르므로 따로 기록합니다. `ignore`를 추가해도 과거 커밋의 `secret.txt` 내용은 남습니다. “목록에 있으니 과거 내용도 삭제됐다”는 답은 틀립니다. 실제 비밀 노출은 담당자에게 알리고 자격 증명 교체와 이력 처리 절차를 확인해야 합니다.

확인 질문과 해설

① 메시지에 왜를 쓰는 이유는 무엇인가요? 다음 검토자가 변경 목적을 알 수 있습니다. ② 생성 파일과 비밀 파일은 같은가요? 둘 다 제외할 수 있지만 제외 이유와 위험은 다릅니다. ③ 작업 상태가 `clean`이면 요구가 맞나요? 기록 여부만 말하므로 요구와 테스트를 따로 확인합니다.

개념. 한 커밋에는 설명할 수 있는 한 목적의 변경을 묶는 것이 좋습니다. 메시지는 나중에 기록을 읽는 사람에게 무엇을 바꿨는지 알려 주는 짧은 설명입니다.

왜 필요한가. 안내 수정과 데이터 삭제를 한 기록에 섞으면 무엇을 검토하거나 되돌려야 하는지 어려워집니다. 파일 개수보다 변경 목적의 일관성을 봅니다.

함께 보는 예. README의 `Run checker`.를 `Run checker from course`.로 바꾸고 ‘Clarify run folder’라고 기록합니다. 문구 변경의 이유가 실행 폴더를 명확히 하기 위함임을 설명할 수 있습니다.

헛갈리는 경우. ‘수정’, ‘최종’만 적으면 어떤 내용인지 알기 어렵습니다. 반대로 메시지에 실제 하지 않은 검증을 썼다고 검증한 것이 되지는 않습니다.

전체 명령

```
git diff
git add README.md
git diff --staged
git commit -m "Clarify run folder"
git log --oneline -2
```

각 줄의 뜻

줄	코드	뜻
1	<code>git diff</code>	작업 변경을 확인합니다.
2	<code>git add README.md</code>	의도한 파일을 선택합니다.
3	<code>git diff --staged</code>	선택한 내용을 다시 봅니다.
4	<code>git commit -m "Clarify run folder"</code>	목적이 드러나는 메시지로 기록합니다.
5	<code>git log --oneline -2</code>	최근 두 기록을 확인합니다.

실행 위치와 확인

현재 위치 `work/git-lab`. 편집기로 README의 Run checker. 한 줄을 Run checker from course.로 바꾸고 저장합니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
<둘째해시> Clarify run folder
<첫째해시> Add checker guide
```

A07 · 따라 하기와 작성칸

A07 · 한 목적의 변경과 메시지

목적: 안내 수정과 데이터 삭제를 한 기록에 섞으면 무엇을 검토하거나 되돌려야 하는지 어려워집니다. 파일 개수보다 변경 목적의 일관성을 봅니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A07_input.md](#)

- [A07_change.txt](#)

- [A07_notes.md](#)

- [A07_change.txt](#)

단계

1. 바꿀 한 줄과 이유를 먼저 씁니다.
2. 저장 후 차이를 검토합니다.
3. 선택·재검토·커밋을 수행합니다.

4. 실제 메시지와 바뀐 줄을 연결합니다.

예상 결과: 안내 한 줄이 바뀐 두 번째 커밋.

확인 방법: 한 커밋에 파일 두 개가 있으면 언제나 잘못된가요?

오류 해결: 예상 밖 줄바꿈 변경이 많으면 편집기 저장 방식과 전체 diff를 확인하고 그 상태로 무작정 기록하지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A08 · 현재 내용과 과거 커밋 비교

개념. HEAD는 현재 체크아웃한 커밋을 가리키는 이름입니다. HEAD~1은 일반적인 한 부모 이력에서 한 단계 전 커밋을 뜻합니다. 특정 시점의 파일을 조회하고 두 커밋을 비교할 수 있습니다.

왜 필요한가. 파일을 현재 상태로 복구하지 않고도 과거 내용을 읽을 수 있습니다. 조회와 변경을 구별하면 필요한 정보를 안전하게 확인할 수 있습니다.

함께 보는 예. 첫 커밋에는 Run checker.가 있고 둘째에는 Run checker from course.가 있습니다. show는 이전 내용을 보여 주고 diff는 둘의 변경 줄을 보여 줍니다.

헷갈리는 경우. HEAD~1을 '어제'라고 외우지 않습니다. 시간 단위가 아니라 이력의 부모 관계이며 첫 커밋만 있으면 이전 커밋이 없습니다.

전체 명령

```
git show HEAD~1:README.md
git diff HEAD~1 HEAD -- README.md
git log --oneline -2
```

각 줄의 뜻

줄	코드	뜻
1	<code>git show HEAD~1:README.md</code>	이전 커밋의 README 내용을 조회합니다.
2	<code>git diff HEAD~1 HEAD -- README.md</code>	이전과 현재 커밋의 해당 파일을 비교합니다.
3	<code>git log --oneline -2</code>	실제 비교한 기록 두 개를 확인합니다.

실행 위치와 확인

현재 위치 work/git-lab. A07까지 커밋 두 개가 있어야 합니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

이전 파일에는 Run checker.
 비교에는 -Run checker. / +Run checker from course.

A08 · 따라 하기와 작성칸

A08 · 현재 내용과 과거 커밋 비교

목적: 파일을 현재 상태로 복구하지 않고도 과거 내용을 읽을 수 있습니다. 조회와 변경을 구별하면 필요한 정보를 안전하게 확인할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A08_input.md](#)
- [A08_history.txt](#)
- [A08_notes.md](#)
- [A08_history.txt](#)

단계

1. 현재 파일을 열어 문구를 확인합니다.
2. 이전 내용을 조회하되 파일이 바뀌지 않았는지 봅니다.
3. 두 커밋의 차이를 자신의 말로 설명합니다.
4. HEAD와 이전 커밋의 관계를 그림니다.

예상 결과: 조회 후 작업 파일은 그대로이고 이전 문구를 읽을 수 있습니다.

확인 방법: show로 과거 파일을 읽으면 작업 파일도 과거로 돌아가나요?

오류 해결: HEAD~1 오류는 커밋 개수를 먼저 봅니다. 이력의 시작에는 부모가 없습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A09 · .gitignore와 관리 대상

개념. .gitignore는 특정 경로의 추적되지 않은 파일을 상태와 추가 대상에서 무시하도록 정하는 규칙입니다. 이미 추적한 파일의 과거 기록을 삭제하는 기능은 아닙니다.

왜 필요한가. 임시 실행 결과나 캐시를 코드·안내 변경과 섞지 않도록 관리 범위를 정합니다. 무시 규칙이 무엇을 포함하는지 직접 확인해야 합니다.

함께 보는 예. scratch.txt를 무시 목록에 적고 임시 메모를 만듭니다. status에서 그 파일이 빠지고 check-ignore로 적용된 규칙을 볼 수 있습니다. .gitignore 자체는 공유할 설정으로 커밋할 수 있습니다.

헛갈리는 경우. 이미 커밋한 파일 이름을 .gitignore에 넣어도 이력에서 사라지지 않습니다. 공개 범위를 숨기기 위한 도구로 오해하지 않습니다.

전체 명령

```
git status --short
git check-ignore -v scratch.txt
git add .gitignore
git commit -m "Ignore scratch notes"
```

각 줄의 뜻

줄	코드	뜻
1	<code>git status --short</code>	어떤 파일이 변경 대상으로 보이는지 확인합니다.
2	<code>git check-ignore -v scratch.txt</code>	임시 파일을 무시한 규칙을 찾습니다.
3	<code>git add .gitignore</code>	무시 규칙 파일만 선택합니다.
4	<code>git commit -m "Ignore scratch notes"</code>	관리 대상 규칙을 기록합니다.

실행 위치와 확인

work/git-lab에 편집기로 .gitignore를 만들고 한 줄 scratch.txt를 씁니다. 임시 scratch.txt도 만듭니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
.gitignore:1:scratch.txt scratch.txt
scratch.txt는 일반 상태 목록에서 제외됩니다.
```

A09 · 따라 하기와 작성칸

A09 · .gitignore와 관리 대상

목적: 임시 실행 결과나 캐시를 코드·안내 변경과 섞지 않도록 관리 범위를 정합니다. 무시 규칙이 무엇을 포함하는지 직접 확인해야 합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A09_input.md](#)

- [A09_ignore.txt](#)
- [A09_notes.md](#)
- [A09_ignore.txt](#)

단계

1. 무시할 임시 파일과 보존할 안내 파일을 고릅니다.
2. 규칙과 임시 파일을 만듭니다.
3. status와 check-ignore를 비교합니다.
4. 이미 추적된 파일과의 차이를 설명합니다.

예상 결과: scratch는 무시되고 .gitignore 규칙이 기록됩니다.

확인 방법: .gitignore가 과거 커밋의 파일을 지우나요?

오류 해결: .gitignore.txt로 저장되었는지 확인합니다. 정확한 파일명과 규칙 경로를 봅니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과-근거: _____

다음에 확인할 것: _____

4교시 · 비교와 범위 있는 복구

14:00~14:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 선택 취소와 내용 복구

먼저 할 활동: A10, A11

추가·복습: A12 · 필수 결과를 확인한 뒤 진행합니다.

A10 · 스테이징 취소와 작업 내용 유지

사례로 이해하기 · 취소하려는 대상부터 정하는 복구

실무 상황

동료가 README를 잘 고쳤지만 아직 검토하지 않아 이번 커밋에서 빼고 싶습니다. 다른 경우에는 연습 파일에 쓴 틀린 문장 자체를 버리고 마지막 기록으로 돌아가고 싶습니다. 둘은 취소 대상이 다르므로 같은 명령을 쓰면 필요한 내용을 잃을 수 있습니다.

쉬운 풀이

스테이징 취소는 제출 봉투에서 사본을 빼는 일입니다. 작업 내용 복구는 책상 위 종이를 이전 사본으로 바꾸는 일입니다. `git restore --staged README.md`는 기본적으로 마지막 커밋을 기준으로 선택 상태를 돌립니다. 작업 파일의 문장은 유지합니다. `git restore --source=HEAD --worktree README.md`는 작업 파일을 마지막 커밋 내용으로 바꿉니다. 기록하지 않은 편집 내용은 사라질 수 있으므로 복사본을 남기고 대상 파일을 명시합니다.

완성 예와 단계 풀이

마지막 기록 문장이 `Run from course`인 연습 저장소에서 README를 `Run from wrong folder`로 바꾸고 add합니다. 먼저 staged diff를 읽고 `git restore --staged README.md`를 실행합니다. status에는 아직 작업 변경이 남습니다. 탐색기나 편집기로 문장도 확인합니다. 잘못된 문장을 버리는 것이 확정되었을 때만 `git restore --source=HEAD --worktree README.md`를 실행합니다. 문장이 다시 `course`로 돌아오고 작업 차이도 사라집니다. 다른 파일에는 이 명령을 확장하지 않습니다.

독립 추가 복습 과제

“이번 제출에서만 빼기”와 “틀린 문장 폐기하기” 두 요청에 명령을 연결하고, 실행 전 남겨야 할 증거를 작성하세요. 아직 커밋하지 않은 새 파일도 같은 방식으로 과거 내용에 복구할 수 있는지 판단하세요.

답과 대표 오답

제출 선택만 취소할 때는 staged 옵션을, 작업 내용을 이전 기록으로 바꿀 때는 source와 worktree를 선택합니다. 새 파일에 과거 기록이 없으면 돌아갈 사본도 없습니다. “restore는 언제나 파일을 안전하게 보관한다”는 오답입니다. 목적과 기준을 읽지 않은 복구는 편집 내용을 버릴 수 있습니다.

확인 질문과 해설

① 취소 뒤 문장이 남는 것은 실패인가요? 선택만 취소했다면 정상입니다. ② HEAD는 무엇인가요? 현재 마지막 커밋을 가리킵니다. ③ 복구 확인은 무엇으로 하나요? status만 보지 말고 파일 내용과 비교 결과를 함께 확인합니다.

개념. `git restore --staged`는 선택한 파일의 스테이징 상태를 현재 커밋 기준으로 되돌립니다. 작업 파일의 편집 내용은 유지합니다. 기본 restore와 대상이 다릅니다.

왜 필요한가. 실수로 선택한 변경을 커밋 전에 빼도 작성한 내용은 계속 보관하고 싶은 경우가 있습니다. 취소 대상이 파일 내용인지 다음 기록의 선택인지 구별해야 합니다.

함께 보는 예. README에 Practice note.를 추가해 add한 뒤 `--staged`로 선택을 취소합니다. staged 비교는 비교 작업 diff에는 문장이 남습니다.

헷갈리는 경우. `git restore README.md`는 작업 파일 내용을 바꾸는 명령입니다. `--staged`가 없는 명령과 같은 결과라고 가정하지 않습니다.

전체 명령

```
git add README.md
git diff --staged
git restore --staged README.md
git diff --staged
git diff -- README.md
```

각 줄의 뜻

줄	코드	뜻
1	<code>git add README.md</code>	현재 변경을 선택합니다.
2	<code>git diff --staged</code>	선택 내용을 확인합니다.
3	<code>git restore --staged README.md</code>	다음 커밋의 선택만 취소합니다.
4	<code>git diff --staged</code>	선택 취소 결과를 확인합니다.
5	<code>git diff -- README.md</code>	작업 파일에 남은 편집을 확인합니다.

실행 위치와 확인

work/git-lab의 README 끝에 Practice note.를 추가해 저장합니다. 현재 내용을 별도 노트에도 기록해 둡니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
staged 차이는 사라집니다.
작업 diff에는 +Practice note.가 남습니다.
```

A10 · 따라 하기와 작성칸

A10 · 스테이징 취소와 작업 내용 유지

목적: 실수로 선택한 변경을 커밋 전에 빼되 작성한 내용은 계속 보관하고 싶은 경우가 있습니다. 취소 대상이 파일 내용인지 다음 기록의 선택인지 구별해야 합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A10_input.md](#)

- [A10_unstage.txt](#)

- [A10_notes.md](#)

- [A10_unstage.txt](#)

단계

1. 취소 전에 작업과 staged 상태를 그립니다.
2. README 한 파일만 선택 취소합니다.
3. 파일을 직접 열어 문구가 유지되는지 확인합니다.

4. 선택 취소와 내용 복구를 문장으로 구별합니다.

예상 결과: 작업 문구 유지, staged 변경 없음.

확인 방법: 스테이징을 취소하면 편집한 문구가 사라지나요?

오류 해결: 복구 명령을 외워 입력하기 전에 옵션과 대상 파일을 읽고 현재 상태를 기록합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A11 · 연습 파일 한 개의 작업 내용 복구

개념. git restore -- README.md는 기본적으로 스테이징 상태를 기준으로 작업 파일을 복구합니다. A10에서 선택을 취소한 현재 상황에서는 최근 커밋의 README 내용으로 돌아옵니다.

왜 필요한가. 연습 변경을 버릴 때 어느 내용을 잃고 어느 기록이 남는지 알고 수행해야 합니다. 전체 폴더가 아닌 한 파일로 범위를 한정하고 현재 내용을 따로 보관합니다.

함께 보는 예. Practice note.를 백업 텍스트에 복사한 뒤 README만 복구합니다. 문구는 작업 파일에서 사라지지만 이미 만든 커밋 이력은 유지됩니다.

헛갈리는 경우. 복구 전에 따로 기록하지 않은 미커밋 작업은 Git 이력으로 되찾을 수 있다고 보장하지 않습니다. 모든 폴더를 강제로 되돌리는 명령으로 확장하지 않습니다.

전체 명령

```
git diff -- README.md
git restore -- README.md
git status --short
git log --oneline -3
```

각 줄의 뜻

줄	코드	뜻
1	git diff -- README.md	복구로 없어질 작업 차이를 확인합니다.
2	git restore -- README.md	지정한 README 작업 내용만 복구합니다.
3	git status --short	남은 변경을 확인합니다.
4	git log --oneline -3	기존 커밋 이력이 유지되는지 봅니다.

실행 위치와 확인

work/git-lab이며 A10을 마친 상태입니다. Practice note.가 들어 있는 현재 README를 work/README_before_restore.txt로 복사해 보관한 뒤 진행합니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

Practice note. 작업 변경이 사라집니다.
기존 커밋은 그대로 유지됩니다.

A11 · 따라 하기와 작성칸

A11 · 연습 파일 한 개의 작업 내용 복구

목적: 연습 변경을 버릴 때 어느 내용을 잃고 어느 기록이 남는지 알고 수행해야 합니다. 전체 폴더가 아닌 한 파일로 범위를 한정하고 현재 내용을 따로 보관합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A11_input.md](#)
- [A11_restore.txt](#)
- [A11_notes.md](#)
- [A11_restore.txt](#)

단계

1. 사라질 문구와 백업 위치를 먼저 씁니다.
2. 연습 저장소 주소를 다시 확인합니다.
3. 한 파일만 복구하고 직접 엽니다.
4. 기록 이력과 작업 내용의 차이를 설명합니다.

예상 결과: README에서 임시 문구가 없어지고 과거 커밋은 유지됩니다.

확인 방법: restore는 새 커밋을 만드세요?

오류 해결: 주소가 실제 프로젝트라면 이 연습을 수행하지 말고 별도 git-lab로 이동합니다. 백업과 대상 파일명을 먼저 확인합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

5교시 · 공유 저장소와 리뷰

15:00~15:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 공유 저장소와 검토 요청

먼저 할 활동: A13, A15

추가·복습: A14 · 필수 결과를 확인한 뒤 진행합니다.

A13 · 원격 이름과 로컬 공유 저장소

사례로 이해하기 · 로컬 기록과 공유 저장소의 시차

실무 상황

한 사람이 안내를 커밋했는데 옆 사람의 폴더에는 옛 안내가 보입니다. 첫 사람의 status는 clean입니다. 오류일까요? 로컬 기록을 만들었어도 공유 장소로 보내지 않았거나 다른 사람이 새 기록을 받지 않았을 수 있습니다. 각 컴퓨터의 상태를 따로 설명해야 합니다.

쉬운 풀이

remote는 공유 저장소 주소에 붙이는 이름입니다. origin은 관례적인 별칭이며 인터넷을 뜻하지 않습니다. push는 내 기록을 공유 저장소로 보내고 fetch는 공유 저장소의 새 기록을 가져옵니다. fetch만으로 작업 파일이 반드시 바뀌지는 않습니다. clone은 새 작업 폴더와 기록을 처음 복사하는 행동입니다. 오늘은 같은 컴퓨터의 bare 저장소를 공유 장소로 사용합니다. bare는 작업 파일 없이 공유용 기록을 보관하는 저장소입니다.

완성 예와 풀이

별도 빈 연습 work 폴더에서 `git init --bare -b main shared.git`를 만들고 git-lab에서 `git remote add origin ./.shared.git`를 실행합니다. `git remote -v`로 주소를 읽습니다. `git push -u origin main`은 로컬 경로에만 기록을 보냅니다. work로 이동하여 `git clone shared.git colleague`를 하면 동료 역할 폴더가 생깁니다. 이후 git-lab의 새 커밋을 push한 뒤 colleague에서 `git fetch origin`, `git log --oneline main..origin/main`을 읽습니다. 동료가 뒤에 있다는 증거를 본 뒤 `git merge --ff-only origin/main`으로 작업본을 갱신합니다. 갈라진 이력이면 자동 성공을 기대하지 않습니다.

독립 추가 복습 과제

“내 로컬에만 새 기록”, “공유 장소에 새 기록”, “동료가 fetch만 완료” 세 상태를 화살표로 그리세요. 각 상태에서 어느 README가 새 문장을 보일지 결정하고 빠진 행동을 쓰세요.

답과 대표 오답

로컬 커밋 뒤에는 내 파일만 최신이고 push 뒤에도 동료 파일은 옛 상태일 수 있습니다. 동료가 fetch만 했다면 비교용 원격 기록은 최신이지만 작업본은 이전일 수 있습니다. “clean이면 공유도 최신”은 근거 없는 답입니다.

확인 질문과 해설

- ① origin은 반드시 GitHub인가요? 주소에 붙인 이름이므로 로컬도 가능합니다. ② push가 PR을 자동 생성하나요? 별도 서비스 작업입니다. ③ 받기 전 확인할 것은 무엇인가요? 내 변경, 공유 주소, 두 기록의 차이를 확인합니다.
- 개념.** remote는 가져오거나 보낼 다른 저장소의 경로에 붙인 이름입니다. origin은 흔한 이름이며 특별한 계정이나 암호가 아닙니다. 실습에서는 같은 컴퓨터의 다른 폴더를 공유 저장소로 사용합니다.
- 왜 필요한가.** 인터넷 계정 없이 push의 역할을 확인하고 로컬 이력과 공유 대상의 차이를 볼 수 있습니다. 실제 GitHub에 게시한 것으로 보고하지 않습니다.
- 함께 보는 예.** bare 저장소는 일반 작업 파일을 편집하는 폴더 없이 Git 기록을 받는 용도로 사용합니다. ../shared.git를 origin으로 연결하고 main을 보냅니다. 네트워크 주소가 아닌 로컬 상대 경로입니다.
- 헛갈리는 경우.** 원격이라는 말이 반드시 인터넷 서버만 뜻하지는 않습니다. 저장소 경로는 로컬일 수도 있습니다. GitHub는 별도의 웹 서비스이며 이번 shared.git와 동일한 서비스 기능이 생기는 것은 아닙니다.

전체 명령

```
git init --bare -b main ../shared.git
git remote add origin ../shared.git
git remote -v
git push -u origin main
```

각 줄의 뜻

줄	코드	뜻
1	git init --bare -b main ../shared.git	main을 기본 브랜치로 둔 로컬 공유 저장소를 만듭니다.
2	git remote add origin ../shared.git	그 로컬 경로에 origin이라는 이름을 붙입니다.
3	git remote -v	보낼 대상 경로를 확인합니다.
4	git push -u origin main	main 기록을 로컬 공유 저장소로 보냅니다.

실행 위치와 확인

- 현재 위치 work/git-lab. 강사가 로컬 경로 ../shared.git인지 확인합니다. 이 활동은 GitHub 접속이나 인터넷 push를 하지 않습니다.
- 대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
origin ../shared.git (fetch)
origin ../shared.git (push)
main 브랜치가 로컬 shared.git에 전달됩니다.
```

A13 · 따라 하기와 작성칸

A13 · 원격 이름과 로컬 공유 저장소

목적: 인터넷 계정 없이 push의 역할을 확인하고 로컬 이력과 공유 대상의 차이를 볼 수 있습니다. 실제 GitHub에 게시한 것으로 보고하지 않습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A13_input.md](#)

- [A13_local_remote.txt](#)

- [A13_notes.md](#)

- [A13_local_remote.txt](#)

단계

1. 로컬 작업 폴더와 공유 폴더를 그림으로 구별합니다.
2. remote -v의 주소가 로컬 경로인지 확인합니다.
3. 보내기 전후 공유 대상의 역할을 설명합니다.
4. GitHub 게시를 수행하지 않았다고 기록합니다.

예상 결과: 로컬 bare 저장소에 main 이력 전달.

확인 방법: origin이라는 이름만 보면 GitHub 주소인지 알 수 있나요?

오류 해결: origin이 이미 있으면 기존 주소를 먼저 확인합니다. 확인 없이 다른 대상 주소로 덮어쓰지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A15 · GitHub와 PR의 역할

개념. Git은 로컬 이력 도구이고 GitHub는 저장소 호스팅과 협업 기능을 제공하는 웹 서비스입니다. Pull request는 변경을 합치기 전에 목적·차이·확인을 제안하고 검토하는 요청입니다.

왜 필요한가. 커밋과 전송만으로 협업 검토가 끝나는 것은 아닙니다. 다른 사람이 이해할 설명과 확인 범위를 함께 제시해야 합니다. 계정이 없어도 제공한 변경 제안 꾸러미로 리뷰를 연습할 수 있습니다.

함께 보는 예. data/pr_proposal.md와 pr_diff.txt에는 실행 폴더를 명확히 하는 제안이 있습니다. 학생은 목적·실제 변경·검증 근거를 읽고 review_response.md에 수락 또는 보완 이유를 씁니다.

헛갈리는 경우. PR은 로컬 Git 명령의 커밋과 동일하지 않고, 요청을 만들었다고 자동으로 병합되었다는 뜻도 아닙니다. 이번 활동은 제공 카드의 모의 검토이며 GitHub에 실제 요청을 만들지 않습니다.

제공 입력

PR 제안: README 실행폴더 안내 추가.

변경: Run checker. → Run checker from course.

확인: 안내문구 대조만 수행, 프로그램 새 실행은 미수행.

A15 · 따라 하기와 작성칸

A15 · GitHub와 PR의 역할

추가 원문·작업 파일:

- [A15_input.md](#)
- [pr_proposal.md](#)
- [pr_diff.txt](#)
- [A15_notes.md](#)
- [review_response.md](#)

목적: 커밋과 전송만으로 협업 검토가 끝나는 것은 아닙니다. 다른 사람이 이해할 설명과 확인 범위를 함께 제시해야 합니다. 계정이 없어도 제공한 변경 제안 꾸러미로 리뷰를 연습할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A15_input.md](#)
- [A15_notes.md](#)

단계

1. 제안의 문제와 바뀐 줄을 연결합니다.
2. 검증했다는 문구가 실제 확인 범위와 맞는지 봅니다.
3. 보완 질문 또는 수락 근거를 작성합니다.
4. 로컬 commit·push·PR·merge 역할을 네 칸으로 나눕니다.

예상 결과: 계정 없는 모의 리뷰와 실제 GitHub 작업을 구별한 검토 기록.

확인 방법: PR 본문에 '모든 테스트 통과'라고 쓰면 그대로 믿어도 되나요?

오류 해결: 화면이 없어서 막히면 자료의 제안·diff·검토 응답을 순서대로 읽습니다. 실제 UI 사용 경험이라고 기록하지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

6교시 · 협업 사례와 인계

16:00~16:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 충돌 의미 결정과 인계

먼저 할 활동: A16, A17, A18

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A16 · 충돌 표식과 내용 결정

사례로 이해하기 · 충돌을 업무 결정으로 해결하기

실무 상황

두 담당자가 같은 안내 줄을 다르게 수정했습니다. Git은 업무상 맞는 문장을 판단하지 못하므로 충돌을 표시합니다. 사람이 요구를 대조하여 결정합니다.

쉬운 풀이

브랜치는 기록 흐름의 이름이고 병합은 흐름을 합치는 일입니다. 같은 위치를 다르게 고치면 자동 결정이 멈춥니다. 일반 merge 충돌의 <<<<<< HEAD 아래는 현재 쪽, ===== 다음은 합치려는 쪽, >>>>>> guide는 끝 표시입니다. 표식을 제거하고 업무 요구에 맞는 최종 문장을 작성합니다.

완성 예와 단계 풀이

P4의 저장소를 이어 쓰지 않습니다. 압축을 푼 course 폴더에서 아래 명령으로 새 work/git-conflict-demo를 만듭니다. 이미 같은 이름이 있으면 새 빈 폴더 이름을 사용합니다.

```
New-Item -ItemType Directory -Path work/git-conflict-demo
Set-Location work/git-conflict-demo

git init -b main

git config --local user.name "Classroom Demo"
git config --local user.email "learner@example.invalid"

Set-Content -Path README.md -Value "Run from here." -Encoding utf8

git add README.md

git commit -m "Initial common instructions"
```

명령은 새 폴더 생성·이동·main 저장소 시작·로컬 작성자 설정·공통 문장 저장·선택·기록 순서입니다. 이후 실행 위치는 계속 work/git-conflict-demo입니다. `git status --short`가 비었는지 확인합니다. `git switch -c guide`로 갈라 README 첫 줄을 `Run from work.`로 저장하고 `add·commit`합니다. `git switch main`으로 돌아가 첫 줄을 `Run from course.`로 저장하고 `add·commit`합니다. `git merge guide`가 충돌하면 `git status --short`에 `UU README.md`가 보입니다. 최종 요구가 “압축을 푼 course 안의 work에서 실행”이면 README 첫 줄을 `Run from extracted course/work.`로 작성하고

세 표식도 제거합니다. `git add README.md` 후 staged diff를 검토하고 `git commit -m "Resolve run folder instructions"`를 합니다. status와 실제 문장을 함께 확인합니다.

독립 추가 복습 과제

양쪽이 센서 경계값을 28과 30으로 바꿨는데 승인된 규칙이 28 이상 주의입니다. 어떤 문장을 남길지 결정하고 확인할 경계 입력 세 개를 작성하세요. 결정을 할 수 없다면 누구에게 무엇을 물어볼지도 쓰세요.

답과 대표 오답

승인된 28을 유지하고 27.9·28·28.1을 대조합니다. 양쪽 숫자를 평균한 29는 승인 근거가 없습니다. 상위 문장을 무조건 고르는 것도 업무 판단을 대신하지 못합니다. 중단하려면 변경을 보관하고 merge 시작 전 상태가 clean인지 확인한 연습에서 `git merge --abort`를 사용할 수 있습니다.

확인 질문과 해설

① UU는 두 파일인가요? 한 파일의 미해결 충돌입니다. ② add가 의미를 검증하나요? 해결한 내용을 선택할 뿐입니다. ③ 해결 후 무엇을 남기나요? 최종 문장, 선택 근거, 재검사 결과를 기록합니다.

개념. 충돌은 두 변경을 Git이 자동으로 합치기 어려운 상태입니다. 표식은 양쪽 내용을 보여 주며 최종 뜻을 결정하는 일은 사람이 해야 합니다.

왜 필요한가. 같은 줄을 서로 다르게 바꾸면 한쪽을 무조건 지우는 것으로 업무 목적을 보존하기 어렵습니다. 어떤 요구를 남길지 먼저 정하고 최종 문장을 만듭니다.

함께 보는 예. 한쪽은 실행 폴더를, 다른 쪽은 결과 파일 확인을 안내했습니다. 최종 문장에 두 뜻을 담을 수 있는지 검토합니다. 제공 conflict_card.txt에서 표식을 제거하고 문장을 합쳐 씁니다.

헛갈리는 경우. 표식만 없으면 충돌 처리의 의미 검토가 끝난 것은 아닙니다. 어느 쪽의 요구가 빠졌는지 확인해야 합니다. 실제 merge 실습은 강사의 새 임시 저장소 시연 또는 선택 경로입니다.

제공 입력

```
<<<<<<< HEAD
Run from course.

=====
Check output JSON after running.

>>>>>>> docs-output
```

A16 · 따라 하기와 작성칸

A16 · 충돌 표식과 내용 결정

추가 원문·작업 파일:

- [A16_input.md](#)

- [conflict_card.txt](#)

- [A16_notes.md](#)

- [conflict_resolution.txt](#)

목적: 같은 줄을 서로 다르게 바꾸면 한쪽을 무조건 지우는 것으로 업무 목적을 보존하기 어렵습니다. 어떤 요구를 남길지 먼저 정하고 최종 문장을 만듭니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A16_input.md](#)

- [A16_notes.md](#)

단계

1. 양쪽 문장이 요구하는 행동을 나눕니다.
2. 두 뜻을 보존한 최종 문장을 작성합니다.
3. 충돌 표식이 남지 않았는지 확인합니다.
4. 시연 또는 수동 카드 해결 중 실제 수행한 방식을 기록합니다.

예상 결과: Run from course and check output JSON after running. 같은 뜻을 보존한 문장.

확인 방법: 충돌이 난 것은 파일이 모두 망가졌다는 뜻인가요?

오류 해결: 원래 요구를 모르면 작성자에게 확인할 질문을 남깁니다. 이름만 보고 어느 쪽이 맞다고 정하지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A17 · 새 안내 변경을 스스로 기록하기

개념. 종합 활동은 작은 새 요구를 받아 변경·비교·선택·기록을 이어 수행하는 일입니다. 이미 배운 명령을 재사용하며 목적과 근거를 설명합니다.

왜 필요한가. 이전 예제의 정답 명령을 따라 한 것과 새 문구를 올바른 범위에 반영하는 것을 구별합니다. 한 목적의 변경과 재현 가능한 증거를 남깁니다.

함께 보는 예. 새 요구는 README에 'Missing values stay unknown.'를 추가해 미측정을 추측하지 않는다는 안내를 보강하는 것입니다. 프로그램 코드의 실제 동작을 바꾸었다고 주장하지 않습니다.

헛갈리는 경우. 문서 변경만 했는데 '검사기 버그를 고쳤다'라고 메시지를 쓰면 내용과 기록이 맞지 않습니다. 메시지는 실제 변경을 설명해야 합니다.

전체 명령

```
git status --short
git diff -- README.md
git add README.md
git diff --staged
git commit -m "Explain missing-value policy"
git log --oneline -1
```

각 줄의 뜻

줄	코드	뜻
1	<code>git status --short</code>	기존 변경이 있는지 봅니다.
2	<code>git diff -- README.md</code>	새 안내 한 줄을 비교합니다.
3	<code>git add README.md</code>	그 파일을 선택합니다.
4	<code>git diff --staged</code>	기록할 최종 차이를 봅니다.
5	<code>git commit -m "Explain missing-value policy"</code>	실제 문서 목적을 메시지로 남깁니다.
6	<code>git log --oneline -1</code>	생긴 기록을 확인합니다.

실행 위치와 확인

work/git-lab로 돌아와 상태를 확인합니다. README에 지정 문장을 추가하고 저장합니다. peer에서 작업하지 않도록 경로를 확인합니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
<새해시> Explain missing-value policy
README에 Missing values stay unknown.가 포함됩니다.
```

A17 · 따라 하기와 작성칸

A17 · 새 안내 변경을 스스로 기록하기

목적: 이전 예제의 정답 명령을 따라 한 것과 새 문구를 올바른 범위에 반영하는 것을 구별합니다. 한 목적의 변경과 재현 가능한 증거를 남깁니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A17_input.md](#)

- [A17_final_change.txt](#)
- [A17_notes.md](#)
- [A17_final_change.txt](#)

단계

1. 새 요구와 수정할 파일을 기록합니다.
2. 지정 한 문장만 추가합니다.
3. 비교·선택·재검토·커밋을 수행합니다.
4. 해시·메시지·실제 차이를 인계 노트에 남깁니다.

예상 결과: 새 안내를 담은 목적이 분명한 커밋.

확인 방법: 기록 뒤 공유 저장소에도 새 커밋이 있다고 말해도 되나요?

오류 해결: 다른 변경이 섞여 있으면 그 목적을 먼저 확인하고 한 파일만 선택했다고 모든 내용이 맞다고 가정하지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A18 · 변경 이력과 확인 범위 인계

개념. 인계는 다음 사람이 현재 상태와 변경 이유를 확인하도록 파일·브랜치·커밋·남은 변경·공유 여부를 설명하는 일입니다. 명령 실행 여부와 결과 해석을 함께 적습니다.

왜 필요한가. '완료'만 전달하면 저장만 했는지, 커밋했는지, 공유했는지 구별할 수 없습니다. 수신자가 어느 상태에서 시작할 지 판단할 근거가 필요합니다.

함께 보는 예. change_report.md에 작업 저장소 경로, 최신 메시지와 해시, 수정 문장, 확인한 diff, 로컬 공유 여부, GitHub 미사용을 적습니다. 새 사람은 로그와 파일을 대조합니다.

햇갈리는 경우. 작업 폴더를 ZIP으로 전달할 때 .git 포함 여부에 따라 이력 전달 여부가 달라집니다. 제출 정책에 맞춰 상태 보고를 작성하고 누락된 이력을 있다고 말하지 않습니다.

제공 입력

인계 칸: 저장소 경로 / 브랜치 / 최신커밋 / 변경 목적 / diff 근거 / 남은 변경 / 공유 대상·상태 / GitHub 수행 여부.

개념 보충 공식 근거 (2026-10-05 확인)

- [Git restore: 기준과 작업·선택 영역](#)
- [Git merge: 충돌과 중단](#)

A18 · 따라 하기와 작성칸

A18 · 변경 이력과 확인 범위 인계

추가 원문·작업 파일:

- [A18_input.md](#)

- [A18_notes.md](#)

- [change_report.md](#)

목적: ‘완료’만 전달하면 저장만 했는지, 커밋했는지, 공유했는지 구별할 수 없습니다. 수신자가 어느 상태에서 시작할지 판단할 근거가 필요합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A18_input.md](#)

- [A18_notes.md](#)

단계

1. status·log·remote 결과로 각 칸을 채웁니다.
2. 짝이 같은 커밋과 문장을 찾는지 확인합니다.
3. 로컬 공유와 GitHub 미사용을 구별합니다.
4. 제출물의 이력 포함 여부를 기록합니다.

예상 결과: 저장·기록·공유를 구별한 재현 가능한 인계.

확인 방법: clean 상태이면 원격도 최신이라고 단정할 수 있나요?

오류 해결: 결과가 불명확하면 추정하지 말고 실제 명령과 출력을 남겨 다음 확인을 요청합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

추가 활동 · 필수 경로를 마친 뒤

아래는 기존 웹의 개념·활동입니다. 수업 시간의 필수 결과를 먼저 확인한 뒤 선택하거나 복습에 사용합니다. 기존 웹의 활동 별 분량 배정은 이번 PDF의 50분 교시 시간표와 별도로 보존된 설계입니다.

A12 · 브랜치와 병합의 흐름

개념. 브랜치는 일련의 커밋을 가리키는 이름입니다. 다른 이름에서 작업하면 기본 흐름과 구별하여 제안을 만들 수 있습니다. 병합은 분리된 변경을 이력에 반영하는 작업입니다.

왜 필요한가. 새 안내를 검토한 뒤 반영하는 협업 흐름을 이해합니다. 초심자 필수는 상태·비교·커밋이며 브랜치 명령은 강사 시연 또는 준비된 사람의 선택 실습으로 둡니다.

함께 보는 예. docs-example 브랜치에서 사용 예를 추가한 뒤 main으로 돌아와 비교합니다. 변경을 반영할지 결정한 뒤 병합합니다. 브랜치는 폴더 전체를 수동 복사한 것과는 다릅니다.

헛갈리는 경우. 브랜치를 만들었다고 파일 변경이 자동 커밋되지는 않습니다. 깨끗한 작업 상태에서 시작하고 어디에 커밋했는지 확인합니다.

전체 명령

```
git switch -c docs-example
git add README.md
git commit -m "Add reading example"
git switch main
git diff main docs-example -- README.md
git merge --no-ff docs-example -m "Merge reading example"
```

각 줄의 뜻

줄	코드	뜻
1	<code>git switch -c docs-example</code>	새 브랜치를 만들고 이동합니다.
2	<code>git add README.md</code>	시연에서 추가한 문구를 선택합니다.
3	<code>git commit -m "Add reading example"</code>	그 브랜치에 기록합니다.
4	<code>git switch main</code>	기본 브랜치로 돌아갑니다.
5	<code>git diff main docs-example -- README.md</code>	들어올 변경을 비교합니다.
6	<code>git merge --no-ff docs-example -m "Merge reading example"</code>	검토한 브랜치를 병합 커밋으로 반영합니다.

실행 위치와 확인

강사 시연 또는 선택 실습. work/git-lab의 작업 상태가 깨끗한지 확인합니다. 브랜치 생성 뒤 README 끝에 Example: S01 22.5.를 넣고 저장합니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
main에 Example: S01 22.5.가 반영됩니다.
브랜치의 커밋과 병합 기록을 볼 수 있습니다.
```

A12 · 브랜치와 병합의 흐름

목적: 새 안내를 검토한 뒤 반영하는 협업 흐름을 이해합니다. 초심자 필수는 상태·비교·커밋이며 브랜치 명령은 강사 시연 또는 준비된 사람의 선택 실습으로 둡니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A12_input.md](#)
- [A12_branch_demo.txt](#)
- [A12_notes.md](#)
- [A12_branch_demo.txt](#)

단계

1. 제공 이력 카드에서 main과 docs-example의 위치를 표시합니다.
2. 시연을 보며 작성·커밋·병합을 구별합니다.
3. 두 브랜치 차이를 설명합니다.

- 4. 선택 실행 여부와 최종 결과를 기록합니다.
- 예상 결과:** 분리된 기록과 반영 결정을 구별합니다.
- 확인 방법:** 브랜치를 만들면 GitHub PR도 자동으로 생기나요?
- 오류 해결:** 전환이 막히면 미커밋 변경 상태를 먼저 확인합니다. 강제 전환으로 밀어붙이지 않습니다.

A14 · clone과 새 기록 받아오기

개념. clone은 저장소의 이력을 새 작업 폴더로 가져옵니다. fetch는 새 기록을 받아 확인하는 단계이며 pull은 보통 가져오기와 현재 브랜치에 통합하기를 연결합니다.

왜 필요한가. 팀원이 같은 자료를 다른 폴더에서 보는 상황을 재현합니다. 같은 파일을 복사하는 것과 이력을 함께 가져오는 것을 비교합니다. 실제 두 사람의 계정을 사용하지 않습니다.

함께 보는 예. shared.git를 peer 폴더로 clone하면 README와 이력을 확인할 수 있습니다. 새 기록이 있으면 fetch 뒤 비교하고 pull --ff-only로 선형으로 반영할 수 있을 때만 진행합니다.

헛갈리는 경우. pull은 항상 모든 상황을 자동으로 해결하는 명령이 아닙니다. 로컬 변경이나 분기된 이력이 있으면 추가 판단이 필요합니다. 오류가 나면 force로 덮지 않습니다.

전체 명령

```
git clone ../shared.git ../peer
Set-Location ../peer
git log --oneline -3
git remote -v
git fetch origin
git pull --ff-only
```

각 줄의 뜻

줄	코드	뜻
1	git clone ../shared.git ../peer	로컬 공유 저장소를 새 peer 폴더로 복제합니다.
2	Set-Location ../peer	다른 작업 사본으로 이동합니다.
3	git log --oneline -3	가져온 이력을 확인합니다.
4	git remote -v	이 사본의 공유 대상 주소를 봅니다.
5	git fetch origin	새 기록을 받습니다.
6	git pull --ff-only	단순 전진이 가능할 때만 현재 브랜치에 반영합니다.

실행 위치와 확인

시작 위치 work/git-lab. ../peer는 새로 만들 폴더입니다. 네트워크 주소를 사용하지 않습니다.

대표 확인 결과(커밋 해시·사용자명 등은 환경마다 다름):

```
peer 폴더에서 같은 README와 이력을 확인합니다.
새 기록이 없으면 Already up to date.가 나올 수 있습니다.
```

A14 · clone과 새 기록 받아오기

목적: 팀원이 같은 자료를 다른 폴더에서 보는 상황을 재현합니다. 같은 파일을 복사하는 것과 이력을 함께 가져오는 것을 비교합니다. 실제 두 사람의 계정을 사용하지 않습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A14_input.md](#)
- [A14_clone.txt](#)
- [A14_notes.md](#)
- [A14_clone.txt](#)

단계

1. clone 전후 파일뿐 아니라 이력도 비교합니다.
2. 현재 위치가 peer로 바뀌었음을 기록합니다.
3. fetch와 반영의 역할을 나눠 설명합니다.
4. 새 기록이 없는 결과도 정상 상황인지 판단합니다.

예상 결과: 별도 작업 사본에 같은 이력. 새 변경이 없으면 최신 상태.

확인 방법: peer에서 편집하면 git-lab 작업 파일도 즉시 바뀌나요?

오류 해결: bare의 기본 브랜치가 맞지 않아 비어 보이면 강사가 shared.git의 HEAD가 main인지 확인합니다. 명령 실패를 성공으로 적지 않습니다.

용어와 코드 기호 찾아보기

Git/GitHub 쉬운 용어

용어	뜻
저장소	프로젝트 파일과 변경 기록을 함께 관리하는 단위
작업 영역	지금 편집하는 파일의 상태
스테이징	다음 커밋에 넣을 변경을 선택하는 단계
커밋	선택한 상태를 설명과 함께 남긴 기록
diff	두 상태에서 달라진 줄을 보여 주는 비교
브랜치	일련의 커밋을 가리키는 이름
원격 저장소	다른 경로에서 가져오거나 보낼 수 있는 저장소
PR	변경을 합치기 전에 제안·검토하는 GitHub의 요청
충돌	변경을 자동으로 합치기 어려워 사람이 결정해야 하는 상태

용어 이름만 외우기보다 활동의 값·파일·출력에서 무엇을 가리키는지 짚습니다. 파일 저장·스테이징·커밋·공유가 각각 어느 상태를 바꾸는지 구별합니다.

오늘 배운 내용 확인하기

Git/GitHub 확인과 평가

18활동의 작성·예상·실제 확인을 분리해 봅니다. 속도나 설치 성공만 평가하지 않습니다.

기준	확인할 증거	보완이 필요한 경우
개념	입력·처리·출력 또는 변경 단계의 역할 설명	결과 이름만 외우고 근거를 말하지 못함
수정	지정한 위치와 변경 전후를 연결	여러 곳을 바꾸어 원인을 설명하지 못함
확인	정상·경계·오류 사례의 예상과 실제 구별	실행 성공을 내용 정확성으로 단정
전달	파일·명령·결과·남은 질문을 재현 가능하게 기록	어느 파일을 사용했는지 불명확

각 기준은 '독립 설명 가능 / 힌트 후 가능 / 다시 연습'으로 기록합니다. 개인 실행 여부는 별도 체크하며 미실행을 실패한 개념 이해로 간주하지 않습니다.

구간별 확인 질문

- 1. status를 열 번 실행하면 커밋이 생기나요?
- 2. 커밋 해시가 교안과 달라지면 실패인가요?
- 3. .gitignore가 과거 커밋의 파일을 지우나요?
- 4. 브랜치를 만들면 GitHub PR도 자동으로 생기나요?
- 5. PR 본문에 '모든 테스트 통과'라고 쓰면 그대로 믿어도 되나요?
- 6. clean 상태이면 원격도 최신이라고 단정할 수 있나요?

제출

work의 활동 노트와 수정 파일, 마지막 종합 작업, 확인한 명령·출력 또는 수동 추적 기록을 제출합니다. 제출 위치는 강사 안내를 따릅니다. 정답 폴더를 학생 결과에 복사하지 않습니다.

개념 보충 전이 문제 · 독립 추가 복습

교시별 필수 설명을 다른 상황에 적용하는 추가 복습입니다. 50분 필수 경로에 모두 포함되어 완수된 것으로 처리하지 않습니다. 답안은 먼저 학생이 작성하고 이후 아래 해설을 대조합니다. 설치·계정 성공 여부와 판단 근거를 분리합니다.

전이 1 · 저장과 기록의 구별

문제: 새 안내.md를 저장하고 status만 세 번 실행했습니다. 파일은 어느 상태이며 다음 이력에 남기려면 어떤 행동이 필요
한가요? 인터넷이 없는 경우도 판단하세요.

해설·확인 기준: 새 파일은 아직 추적되지 않은 상태이며 add로 내용을 선택하고 commit으로 기록합니다. status는 조회
입니다. 인터넷 없이도 가능합니다. 저장과 이력과 공유를 셋으로 구별하면 충족입니다.

전이 2 · 세 상태의 값

문제: 커밋=28, 스테이징=30, 작업 파일=29입니다. 기본 diff와 staged diff의 변화 및 지금 커밋할 값을 쓰세요. 업무 요
구 29를 기록할 다음 행동은 무엇인가요?

해설·확인 기준: 기본 diff는 30→29, staged diff는 28→30이며 커밋에는 30이 들어갑니다. 29를 다시 add하고
staged diff를 읽습니다. 마지막 저장값이 무조건 커밋된다는 답은 재학습입니다.

전이 3 · ignore의 시간 범위

문제: 이미 기록한 가짜 secret.txt를 .gitignore에 적었습니다. 과거 커밋과 추적 여부가 어떻게 되나요? 앞으로 실제 비밀
을 다룰 때 보고에 무엇을 남기나요?

해설·확인 기준: 과거 내용과 이미 추적한 상태는 자동 삭제되지 않습니다. 실제 비밀을 연습 자료로 쓰지 않고 노출 발견 시
담당 절차와 자격 증명 교체를 확인합니다. ignore를 이력 삭제라고 말하면 오답입니다.

전이 4 · 복구 대상 선택

문제: README의 좋은 새 문장을 유지하면서 이번 커밋에서만 빼려 합니다. 어떤 restore 옵션을 쓰나요? 마지막 기록 문
장으로 작업 파일까지 돌리는 요청과 어떻게 다르나요?

해설·확인 기준: git restore --staged README.md는 선택을 취소하고 작업 문장을 유지합니다. git restore
--source=HEAD --worktree README.md는 작업 문장을 이전 기록으로 바꾸므로 편집 내용을 보관하고 사용합니다.
대상·기준·사라지는 내용 설명이 필요합니다.

전이 5 · 공유 시차

문제: 내 로컬은 clean이고 새 커밋도 있습니다. 동료 README는 옛 문장입니다. 공유 장소에 보내기와 동료가 받아 적용
하기를 구분하여 가능한 원인 두 개와 확인 방법을 쓰세요.

해설·확인 기준: push 전이거나 동료가 아직 fetch/적용하지 않았을 수 있습니다. remote 주소, 로컬/원격 기록 차이, 동료
작업 상태를 확인합니다. fetch는 곧바로 작업 파일 변경을 보장하지 않습니다. clean으로 공유 상태를 단정하지 않으면 충
족입니다.

전이 6 · 충돌의 업무 판단

문제: 양쪽 브랜치가 경계값을 28과 30으로 수정했고 승인된 요구는 28 이상 주의입니다. 최종 값과 검사 세 개, 해결 절차
를 쓰세요.

해설·확인 기준: 28을 유지하고 27.9→정상, 28→주의, 28.1→주의를 확인합니다. 표식을 제거해 최종 문장을 작성하고
add→staged diff 검토→commit으로 기록합니다. 양쪽 평균이나 위쪽 자동 선택은 요구 근거가 없습니다.

파일과 다음 수업

링크를 눌러 파일을 내려받습니다. 예제 ZIP은 압축을 풀고 README부터 읽습니다. 원본 code·data는 보존하고 work의 작업본을 수정합니다. 파일이 저장된 폴더와 확장명을 확인하세요.

실습 예제 코드와 데이터

Git/GitHub PPT

처음 시작 안내

용어와 코드 기호

학생용 교안

학생 실습 안내

확인 문제

학생 교안 PDF · 내용 보강본

설치·로그인·장비 준비와 학습 성취는 구분합니다. 이 교안의 배정 시간은 실제 초심자 리허설 결과가 아닙니다. 코드·제공 출력·실물 검증의 한계는 해당 웹 수업과 교안 설명을 따릅니다.